

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 1: TESTE DE DESEMPENHO COM APROXIMAÇÃO DE π

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 11/08/2026.

1. EXPERIMENTO

"Implemente um programa em C que calcule uma aproximação de n usando uma série matemática, variando o número de iterações e medindo o tempo de execução. Compare os valores obtidos com o valor real de n e análise como a acurácia melhora com mais processamento. Reflita sobre como esse comportamento se repete em aplicações reais que demandam resultados cada vez mais precisos, como simulações físicas e inteligência artificial."

2. SÉRIE MATEMÁTICA

O algoritmo desenvolvido baseia-se na série infinita de Gregory-Leibniz para o cálculo aproximado da constante π. A metodologia adota a seguinte formulação matemática:

$$\pi = \frac{4}{1} - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} \ldots \quad \text{ou} \quad \pi = 4 \sum_{i=0}^{n-1} \frac{(-1)^i}{2i+1}$$

3. CÓDIGO

A implementação foi desenvolvida na linguagem C, priorizando a eficiência e o controle direto sobre os tipos de dados.

O código está estruturado em dois blocos principais, sendo eles as funções:

- **calcular_valor_pi:** responsável por computar o somatório. Recebe como parâmetro o número de iterações, tipado como unsigned long long para suportar entradas na casa dos bilhões, essenciais para testes de estresse da CPU. O laço de repetição calcula o denominador ímpar (2.0 * i + 1.0) e alterna as operações de adição e subtração por meio do operador módulo (i % 2 == 0)).
- Função Principal (**main**): implementada para receber o número de iterações via linha de comando (argc e argv), conferindo dinamicidade aos testes sem a necessidade de recompilação. A conversão da entrada é feita de forma segura utilizando a função strtoull().

Por fim, ainda na função “main”, após a chamada da função “calcular_valor_pi”, para quantificar a precisão do modelo matemático em relação ao esforço computacional, o algoritmo calcula o erro absoluto. Isso é feito comparando o valor gerado ao final do laço com uma constante “π” de alta precisão (20 casas decimais) definida em tempo de compilação, utilizando a função “fabs()” da biblioteca “<math.h>”.

Para compilação, a fim de extrair o máximo desempenho da arquitetura durante a execução sequencial, a compilação emprega o compilador GCC utilizando a norma C11 (-std=c11) e a flag de otimização -O2.

A aferição do tempo de processamento para diferentes cargas de trabalho é realizada externamente pelo utilitário time do sistema operacional (ex: time ./main), conforme orientado em sala de aula e garantindo uma medição padronizada do tempo real (wall-clock time ou tempo de relógio) e do tempo de CPU gasto em modo usuário.

Figura 1: Código utilizado para testes

```c
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define PI 3.14159265358979323846

// pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - 1/11 + ...
static double calcular_valor_pi(unsigned long long iteracoes) {
    double soma = 0.0;
    for (unsigned long long i = 0; i < iteracoes; ++i) {
        double termo = 1.0 / (2.0 * i + 1.0);
        if (i % 2 == 0) {
            soma += termo;
        } else {
            soma -= termo;
        }
    }
    return 4.0 * soma;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Uso: %s <quantidade_iteracoes>\n", argv[0]);
        return 1;
    }

    // o strtoull() converte uma string para um unsigned long long
    const unsigned long long quantidade_iteracoes = strtoull(argv[1], NULL, 10);
    const double valor_calculado = calcular_valor_pi(quantidade_iteracoes);
    const double erro_absoluto = fabs(valor_calculado - PI);

    printf("Iteracoes: %llu\n", quantidade_iteracoes);
    printf("- Valor calculado: %.15f\n", valor_calculado);
    printf("- Erro absoluto: %.15f\n", erro_absoluto);

    return 0;
}
```

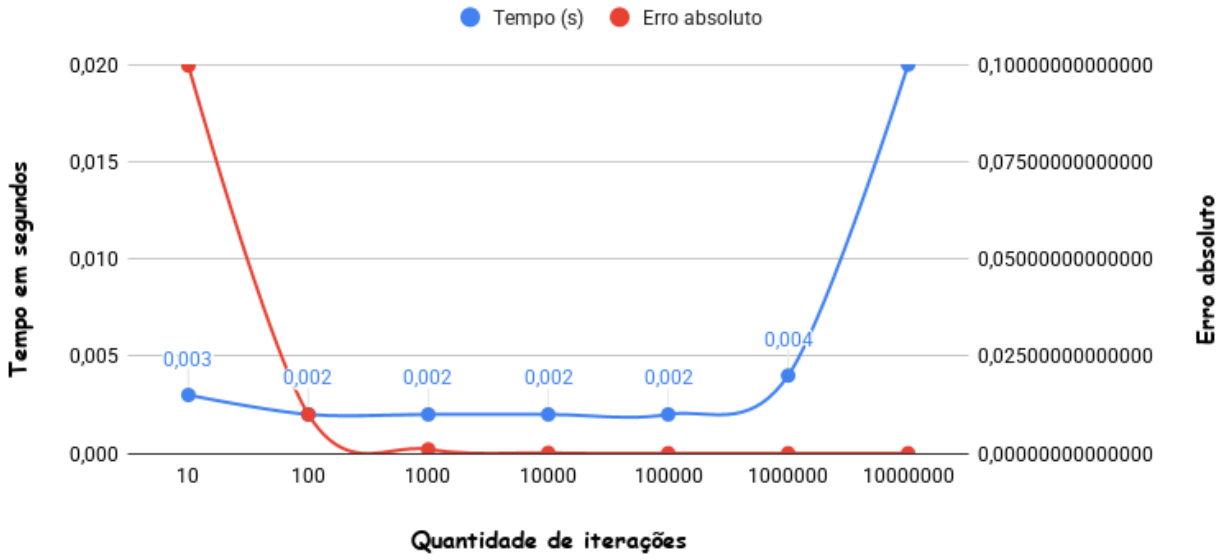
4. RESULTADOS

O resultado dos testes do programa gerado pelo código acima, podem ser vistos na tabela (Tabela 1) e gráfico abaixo (Figura 2).

Tabela 1: Dados do experimento

Quantidade de iterações	Tempo (s)	Erro
10	0,003	0,09975303466039
100	0,002	0,00999975003124
1000	0,002	0,00099999975000
10000	0,002	0,00009999999976
100000	0,002	0,00001000000007
1000000	0,004	0,00000100000002
10000000	0,020	0,00000010000000
100000000	0,172	0,00000001000047
1000000000	1,809	0,00000000100174
10000000000	18,541	0,00000000010145

Figura 2: Evolução do erro e do tempo ao longo das iterações



Com tais dados, é possível constatar que nas primeiras execuções (até 100.000 iterações), o tempo fica quase travado entre 0,002 e 0,003 segundos. Entretanto, esse tempo não é do cálculo matemático em si, mas sim do esforço do

sistema operacional para iniciar o programa, organizar a memória e imprimir os textos na tela. Já a partir de 1 milhão de iterações, o tempo real de processamento começa a ficar evidente. Pode-se observar que sempre que a quantidade de iterações é multiplicada por 10, o tempo de execução também cresce cerca de 10 vezes (passando de 0,172s para 1,809s, e depois chegando a 18,541s). Isso mostra na prática que o código tem um tempo de execução linear.

Ao observar a coluna de “Erro”, notamos a principal desvantagem dessa série matemática. Para conseguir apenas mais uma casa decimal de precisão (ou seja, diminuir o erro em 10 vezes), o programa exige 10 vezes mais repetições.O speso da alta precisão: Para alcançar um erro muito pequeno, o processador precisou “suar” e rodar 10 bilhões de iterações iterativas.

5. CONSIDERAÇÕES FINAIS

Ao comparar os valores gerados pelo algoritmo com o valor real da constante, fica evidente que a acurácia melhora diretamente com o aumento do esforço computacional. No entanto, para conquistar uma precisão mais refinada (mais casas decimais exatas), o tempo de processamento da CPU cresce de forma drástica.

Esse comportamento do código é um reflexo exato dos desafios enfrentados no mercado de tecnologia. Em aplicações como o treinamento de modelos de inteligência artificial ou a execução de simulações físicas complexas (como na simulação de precisão para sistemas de controle e circuitos eletrônicos; previsão do tempo e modelagem climática; e análise estrutural e redes complexas), a busca por resultados exatos esbarra rapidamente no limite do tempo aceitável.

É justamente nesse gargalo que o modelo sequencial se torna inviável. A necessidade de processar bilhões de cálculos em tempo hábil comprova que a adoção de técnicas de programação paralela não é apenas uma otimização opcional, mas uma necessidade absoluta para viabilizar os avanços nessas áreas.